

Trabajo Práctico 1 – Seminario de Actualización Migración de la base de datos Sakila: MySQL a Microsoft SQL Server

Informe – Actividad 2: Carga de Datos (DML)

Método utilizado

Se optó por adaptar el script de INSERTs original de Sakila (sakila-data.sql) en lugar de usar el SQL Server Import and Export Wizard, para tener control total sobre la transformación de nombres y evitar errores de tipeo en ~46.000 registros.

El proceso de adaptación consistió en:

1. **Traducción de nombres de tabla:** cada INSERT INTO tabla_ingles se reemplazó por INSERT INTO tabla_español, siguiendo el mismo mapeo definido en el DDL (customer : cliente, film : película etc.). Los valores de cada fila no se modificaron.
2. **Eliminación de sintaxis exclusiva de MySQL** no soportada en T-SQL: SET AUTOCOMMIT, SET FOREIGN_KEY_CHECKS, LOCK TABLES / UNLOCK TABLES, COMMIT sueltos y los CREATE TRIGGER intercalados en el dump (fuera del alcance de esta actividad).
3. **Listas explícitas de columnas** en cada INSERT: SQL Server exige declarar las columnas cuando se usa IDENTITY_INSERT ON, a diferencia de MySQL donde no es necesario.
4. **SET IDENTITY_INSERT [tabla] ON/OFF** alrededor de cada INSERT, ya que los datos traen los IDs originales explícitos y las columnas correspondientes son IDENTITY en SQL Server.
5. **División de INSERTs grandes en lotes de 1000 filas:** SQL Server limita a 1000 el número de filas por sentencia INSERT ... VALUES (a diferencia de MySQL, que no tiene ese límite). Las tablas pelicula_actor, inventario, pago y alquiler se dividieron en múltiples sentencias manteniendo el total de registros original.
6. **Manejo de dependencias entre tablas:** como el dump original inserta las tablas en un orden que no respeta las relaciones de clave foránea (por ejemplo address antes que city), se deshabilitaron todas las restricciones de FK antes de la carga con sp_MSforeachtable "ALTER TABLE ? NOCHECK CONSTRAINT ALL", y se volvieron a habilitar y validar al finalizar con WITH CHECK CHECK CONSTRAINT ALL. Esto equivale al SET FOREIGN_KEY_CHECKS=0 usado en el script original de MySQL.

No se migró la tabla pelicula_texto (film_text), ya que en el original se completa mediante un trigger sobre film y no tiene INSERTs propios en el dump; tampoco se replicaron los triggers, por estar fuera del alcance de esta actividad.

Captura de pantalla con el conteo de filas de las tablas principales:

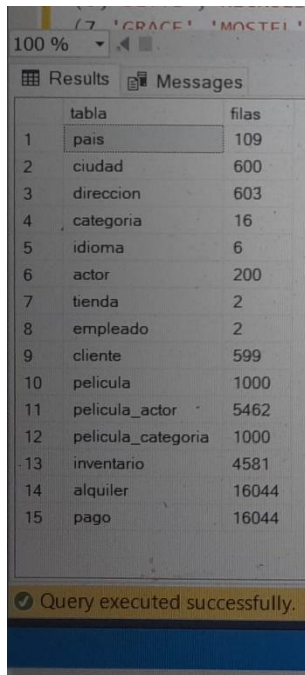


	tabla	filas
1	pais	109
2	ciudad	600
3	direccion	603
4	categoria	16
5	idioma	6
6	actor	200
7	tienda	2
8	empleado	2
9	cliente	599
10	pelicula	1000
11	pelicula_actor	5462
12	pelicula_categoria	1000
13	inventario	4581
14	alquiler	16044
15	pago	16044

Query executed successfully.

1. Conclusiones generales

La migración de Sakila de MySQL a SQL Server permitió comprobar que, más allá de que ambos son motores relacionales, las diferencias no son solo de sintaxis sino también de comportamiento: tipos de datos sin equivalente directo, reglas de integridad referencial más estrictas, y límites operativos (como el máximo de filas por INSERT) que no existen en MySQL.

El esquema resultante conserva la misma cardinalidad y las mismas relaciones que el original, con 15 de las 16 tablas migradas con datos (se excluyó pelicula_texto, dependiente de triggers no requeridos) y con la totalidad de los ~46.000 registros originales, verificados fila por fila contra el dump de origen.

3. Dificultades encontradas

3.1 Cascada de claves foráneas (previsto por la consigna)

Al mantener ON UPDATE CASCADE como en el original, SQL Server rechazó la creación de las FK con el error *"may cause cycles or multiple cascade paths"* en tres puntos:

- Ciclo directo entre tienda y empleado (la tienda referencia a su gerente, el empleado referencia a su tienda).
- Dos caminos en cascada desde pelicula hacia idioma (idioma normal e idioma original).
- Dos caminos en cascada hacia direccion desde cliente y empleado (uno directo, otro vía tienda).

Decisión y justificación: se reemplazó ON UPDATE CASCADE por NO ACTION en la gran mayoría de las FK. Los identificadores son claves subrogadas IDENTITY que en la práctica nunca se actualizan, por lo que el cascade no aportaba ningún beneficio funcional real — solo generaba conflictos estructurales en SQL Server. Se mantuvo ON DELETE SET NULL únicamente en pago → alquiler, porque ahí sí tiene sentido de negocio: conservar el historial de pagos aunque se elimine el alquiler que le dio origen.

3.2 Referencia circular tienda : empleado

Además del problema de cascada, tienda y empleado se referencian mutuamente y no pueden crearse en un único CREATE TABLE sin que una de las dos FK ya exista. Se resolvió creando tienda sin la columna de FK hacia empleado, creando luego empleado, y agregando la FK faltante al final con ALTER TABLE ... ADD CONSTRAINT.

3.3 Columnas explícitas obligatorias con IDENTITY_INSERT ON (no previsto originalmente)

MySQL permite INSERT INTO tabla VALUES (...) sin listar columnas incluso con AUTO_INCREMENT. SQL Server, en cambio, exige declarar explícitamente la lista de columnas cuando se usa SET IDENTITY_INSERT ON para poder insertar los IDs originales del dump. Se agregó la lista de columnas correspondiente a cada INSERT.

3.4 Orden de carga no respeta las dependencias de FK

El dump original inserta las tablas en un orden pensado para MySQL con SET FOREIGN_KEY_CHECKS=0 (por ejemplo, address antes que city y country). Se resolvió deshabilitando todas las restricciones de FK antes de la carga (ALTER TABLE ... NOCHECK CONSTRAINT ALL) y volviendo a habilitarlas y validarlas al finalizar (WITH CHECK CHECK CONSTRAINT ALL), replicando el mismo criterio del script original de forma nativa en T-SQL.

3.5 Columna GEOMETRY condicional

El campo location de address está envuelto en comentarios condicionales de MySQL (/*!50705 ... */), activos solo desde MySQL 5.7.5) y no forma parte de los tipos de datos que pide adaptar la consigna. **Decisión:** no se migró esa columna al esquema en español. Como consecuencia práctica, al no requerir conversión, esos comentarios se comportan como comentarios estándar de SQL Server (/* */) y el dato queda descartado automáticamente sin intervención adicional sobre el dump de datos.

4. Traducción del esquema

Se tradujeron todos los nombres de tablas y columnas al español (customer : cliente, film : película, rental_date : fecha_alquiler, etc.), respetando el convenio de prefijo id_ para claves primarias y foráneas (id_cliente, id_pelicula, etc.) para mantener consistencia y legibilidad en todo el modelo.

Decisión sobre tildes y ñ: se evaluó usar identificadores con tildes y ñ (ej. película, año, descripción) para mayor naturalidad del español, pero finalmente se optó por **omitir los acentos en los identificadores** (pelicula, anio_estreno, descripcion, contrasena), manteniéndolos únicamente en los datos y en la documentación. Motivo: los nombres de tabla y columna son metadatos que se referencian constantemente en scripts, herramientas de terceros y configuraciones de conexión; los caracteres acentuados pueden generar problemas de collation o de codificación al portar el script entre distintas herramientas (SSMS, Azure Data Studio, drivers de otros lenguajes), mientras que evitarlos no genera ninguna pérdida de legibilidad relevante.

